

A dynamic provenance graph-based detector for advanced persistent threats

Lin Wang^a, Lanting Fang^{b,*}, Yining Hu^{a,c}

^a School of Cyber Science and Engineering, Southeast University, 211189, Nanjing, China

^b School of Computer Science and Technology, Beijing Institute of Technology, 100081, Beijing, China

^c Jiangsu Provincial Key Laboratory of Computer Network Technology, 211189, Nanjing, China

ARTICLE INFO

Dataset link: <https://github.com/sbustreamspot/sbustreamspot-data>, <https://github.com/margoseltzer>

Keywords:

Advanced Persistent Threats
Provenance graph
Graph representation learning
Attack detection

ABSTRACT

Advanced Persistent Threats (APTs) pose a major cyber threat due to their stealthy, long-term nature and intricate complexity, making them particularly challenging to detect. Provenance graphs map interactions between system entities as directed, heterogeneous networks, offering rich semantic information valuable for threat identification. However, most existing approaches rely on static graph analysis, overlooking the critical dynamics of evolving threats. Dynamic graph analysis offers the potential to capture both temporal and structural insights, but current methods focus on single-perspective learning, failing to fully exploit the inherent relationships and evolving patterns within the data. To address this gap, we propose CGL-AD, a Contextualized Graph Learning APT Detector. CGL-AD leverages temporal graph learning to capture subtle temporal changes and overall structural transformations over time. It then integrates hash-based data stream frequency estimation technique to identify local topological alterations, subsequently feeding these rich embeddings into a powerful sequence learning model. Experimental results on three widely used datasets: Streamspot, Camflow-apt, and Shellshock, demonstrate that CGL-AD significantly outperforms existing methods. Specifically, CGL-AD outperforms the best baseline FLASH on these datasets by 1.5%, 3.7%, and 5.6% respectively in terms of Receiver Operating Characteristic-Area Under the Curve (ROC-AUC), effectively revealing hidden APT attack patterns in dynamic provenance graphs.

1. Introduction

Advanced Persistent Threats (APTs) constitute a pervasive and multifaceted threat landscape within the cyberspace domain. Defined by their complexity and multi-stage execution, APTs are the preferred tools of highly skilled adversaries targeting high-value entities such as governments, military installations, and financial institutions (Zipperle, Gottwalt, Chang, & Dillon, 2022). Consequently, the detection of APTs emerges as a crucial challenge.

Traditional intrusion detection systems (Dornhackl, Kadletz, Luh, & Tavolato, 2014; Wagner et al., 2015) typically analyze native system logs or rely on malware signatures. However, APT attackers often act in a stealthy and persistent manner, exploiting zero-day vulnerabilities to gain a foothold in the victim system, while remaining undetected for extended periods. Traditional detection methods either fail to recognize new vulnerabilities or focus primarily on log-adjacent events.

In recent studies, researchers have emphasized the use of provenance graphs for APT detection (Jenkinson et al., 2017; Pohly, McLaughlin, McDaniel, & Butler, 2012; Xie, Feng, Tan, & Zhou, 2016). Provenance graphs represent system execution as directed, heterogeneous graphs that describe information flow between system

entities. As shown in Fig. 1, the provenance graph depicts system entities (e.g., processes, files, sockets, etc.) as vertices, and system calls between entities (e.g., create, read, write, open, etc.) as edges. In this example, the attacker exploits the Firefox backdoor vulnerability to launch an attack on an Ubuntu host through a malicious advertising server (i.e. untrustworthy IP 104.228.117.212). This exploit results in the drakon implant running in memory within the Firefox process, establishing a connection to the attacker's operator console. The attacker then writes executable binaries to the host's disk and executes the *clean* and *profile* processes as root through privilege escalation. The entire attack may be concealed among a large number of benign system calls and can unfold in multiple stages over an extended period. Importantly, the provenance graph links related events even when they are temporally separated. However, the complexity and diversity of interactions in provenance graphs makes it challenging to learn patterns of system behavior.

Existing provenance-based detectors can be classified into three categories: (1) Rule-based methods (Hossain et al., 2017; Milajerdi, Eshete, Gjomemo and Venkatakrishnan, 2019; Milajerdi, Gjomemo,

* Corresponding author.

E-mail addresses: 220224982@seu.edu.cn (L. Wang), ltfang@bit.edu.cn (L. Fang), hyn.list@seu.edu.cn (Y. Hu).

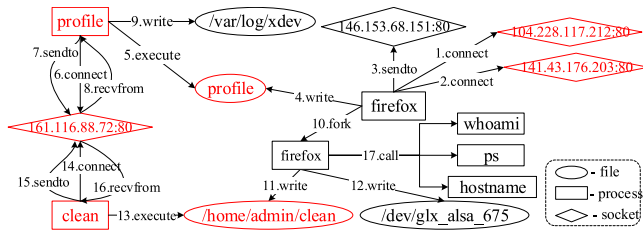


Fig. 1. An example provenance graph of a real-world APT attack, where red nodes are malicious entities, numbers indicate timestamps of system calls.

Eshete, Sekar and Venkatakrishnan, 2019) detect APTs by manually designing a variety of rules based on threat knowledge from known attack patterns. (2) Statistical-based methods (Hassan et al., 2019, 2020; Liu et al., 2018) assess the suspiciousness of audit records based on their rarity within the provenance graph. (3) Learning-based methods (Chen et al., 2022; Kapoor, Melton, Ridenhour, Krishnan, & Moyer, 2021; Wang et al., 2022; Zengy et al., 2022) use deep learning techniques (i.e., graph neural networks) to learn node feature in provenance graphs and detect anomalies.

Although existing methods have made considerable progress in detecting APTs, they suffer from various combination of the following challenges: (1) Heuristic rule-based (Hossain et al., 2017; Milajerdi, Eshete et al., 2019; Milajerdi, Gjomemo et al., 2019; Xiong et al., 2020) methods rely on prior knowledge of APTs and are particularly vulnerable to unknown attacks. (2) Static graph learning methods (Jia et al., 2023; Wang et al., 2022; Zengy et al., 2022) overlook the importance of temporal information, and they lack the ability to model the dynamic behavioral patterns of runtime systems. (3) Recently, dynamic graph learning methods (Cheng et al., 2023; Paudel & Huang, 2022; Rehman, Ahmadi, & Hassan, 2024; Yang, Xu, Xiong, Li, & Zhang, 2023) have been proposed to automatically capture the contextual information from provenance graph. However, these methods overlook the critical relationship between the local subgraphs and the global graph, which is essential for effective APT detection. For example, attackers often adopt a gradual infiltration strategy when stealing data, with initial intrusion activities possibly confined to a small area before expanding to the entire network. Attack behaviors at different stages may impact the local subgraph and global graph structure in different ways. Therefore, it is essential to consider the pattern changes in both local subgraphs and the global structure comprehensively.

In this work, we address the aforementioned challenges by proposing CGL-AD, a **C**ontextualized **G**raph **L**earning **A**PT **D**etector. CGL-AD captures contextual information through a multi-perspective approach. Specifically, CGL-AD employs a temporal graph network architecture to encode both temporal and node history information into node embeddings, learning global structural transformations over time. Simultaneously, it incorporates the frequency of local substructures within the global graph, effectively capturing interaction patterns between heterogeneous entities from a local perspective. Moreover, instead of extracting isolated data points for clustering and outlier analysis as most existing methods (Han, Pasquier, Bates, Mickens, & Seltzer, 2020; Jia et al., 2023; Manzoor, Milajerdi, & Akoglu, 2016) do, we introduce a sequence learning model, Bi-RCNN, to gain a deeper understanding of the system's evolutionary state. The model learns normal behavioral pattern changes by maximizing the co-occurrence relationship between neighboring graph sketches, then identifies anomalies based on predicting deviations in the sequence. To the best of our knowledge, we are the first to simultaneously capture temporal dynamics, global structural information and statistical feature of local substructures. Experimental results also demonstrate that CGL-AD significantly improves the effectiveness of APT attack detection.

The main contributions of this paper are summarized as:

- We propose a novel unsupervised anomaly detection framework for APTs based on temporal graph learning. This approach leverages the rich temporal dynamics and structural relationships within provenance graphs to effectively identify anomalous activities indicative of APT intrusions.
- We introduce CGL-AD, an innovative amalgamation of temporal graph learning, advanced data stream frequency estimation technique and sequence learning. This proficiency enables a more refined and comprehensive characterization of system entity behaviors, leading to improved APT detection accuracy.
- We implemented CGL-AD and evaluated it on three public datasets. The results show that CGL-AD achieved high accuracy and a low false positive rate in APT attacks detection.

2. Related work

The existing Provenance-based APT detection approaches can be categorized into rule-based, statistical-based, and deep learning-based methods. We summarize these studies in Table 1, including algorithms for embedding and threat detection, as well as their limitations.

2.1. Rule-based methods

Rule based methods (Hossain et al., 2017; Milajerdi, Eshete et al., 2019; Milajerdi, Gjomemo et al., 2019; Xiong et al., 2020) detect threats by designing heuristic rules based on prior knowledge of known attacks (e.g., MITRE ATT&CK (Strom et al., 2018)). For instance, Holmes (Milajerdi, Gjomemo et al., 2019) refers to kill chain model (Yadav & Rao, 2015) and a security policy knowledge base, formulating 16 TTP (Tactics, Techniques, Procedures) rules to detect vulnerabilities present in provenance graphs and map alerts to specific attack phases. Poirot (Milajerdi, Eshete et al., 2019) extracts threat query graphs from Cyber Threat Indicator (CTI) reports, and applies a graph alignment algorithm to calculate the similarity between query graph and provenance graph to detect cyber-attacks with known patterns. Although rule-based methods often have a low false alarm rate, designing these rules is an extremely difficult task, as it inevitably requires domain knowledge and expert experience. Moreover, these methods struggle to identify unknown or latent attacks because the rules are designed under specific conditions (e.g., the attack strategies, the network environment). To address these issues, CGL-AD uses a learning-based technique to build detection models without the need for domain knowledge. In addition, the model can be automatically adapted to new conditions by retraining or fine-tuning.

2.2. Statistical-based methods

In statistical-based methods, some approaches (Hassan et al., 2019, 2020; Liu et al., 2018) record the frequency of historical events and assign anomaly scores based on the rarity of edges in provenance graphs. However, they only consider first-order connections between entities, and simple statistical analysis may mistakenly flag rare but normal system activities, such as system upgrades, periodic backups and cleanups, as anomalies. Some studies consider higher-order structural connectivity. For instance, ProvDetector (Wang et al., 2020) identifies the rarest k-hop paths, transforms these paths into vectors using doc2vec, and applies the Local Outlier Factor (LOF) method to identify concealed malicious software. Streamspot (Manzoor et al., 2016) and UNICORN (Han et al., 2020) statistically calculate the frequency of multi-hop local substructures, compress them into low-dimensional graph vectors using a hash function, and use clustering to identify anomalies. However, these methods only consider historical system execution from a localized perspective, and traditional models based on density or distance metrics may not adequately capture relationships between data points. These issues make them vulnerable to

Table 1

A summary of provenance-based APT detection approaches, where ‘-’ indicates that the system does not utilize that module or information for threat detection.

Category	System	Algorithm		Knowledge of attacks	Temporal dynamic	Global context	Local heterogeneous context
		Embedding	Detection				
Rule-based	Holmes (Milajerdi, Gjomemo et al., 2019)	-	TTP rules, Policy Matching	Yes	No	-	-
	Poirot (Milajerdi, Eshete et al., 2019)	-	CTI, Graph Alignment	Yes	No	-	-
	Conan (Xiong et al., 2020)	-	Customized State Transition rules	Yes	No	-	-
Statistical-based	PrioTracker (Liu et al., 2018)	-	Edge rarity, Hill Climbing	No	No	No	Yes
	ProvDetector (Wang et al., 2020)	Path rarity, Doc2vec	LOF	No	No	No	Yes
	Streamspot (Manzoor et al., 2016)	K-shingle, SimHash	K-medoids	No	Yes	No	Yes
	Unicorn (Han et al., 2020)	Frequency Estimation	K-medoids	No	Yes	No	Yes
Deep learning-based	Prov-Gem (Kapoor et al., 2021)	GCN	MLP	Yes	No	Yes	No
	APT-KGL (Chen et al., 2022)	HGAT	RGCN	Yes	No	Yes	Yes
	Shadewatcher (Zengy et al., 2022)	TransR	Dot-product Similarity	No	No	No	Yes
	Threatrace (Wang et al., 2022)	GraphSAGE	Softmax Regression	No	No	Yes	No
	Pikachu (Paudel & Huang, 2022)	Skip-Gram, GRU	Softmax Regression	No	No	Yes	No
	MAGIC (Jia et al., 2023)	Graph Masked AE	KNN	No	No	Yes	No
	PROGRAPHER (Yang et al., 2023)	Graph2vec	TextRCNN	No	Yes	Yes	No
	Kairos (Cheng et al., 2023)	TGN	MLP, IDF	No	Yes	Yes	No
	Flash (Rehman et al., 2024)	Word2vec, GraphSAGE	XGBoost	No	Yes	Yes	No
	CGL-AD(ours)	TGN, Frequency Estimation	Bi-RCNN	No	Yes	Yes	Yes

noise, which leads to false positives. To address these limitations, CGL-AD integrates a dynamic GNN to learn global structural information. Additionally, CGL-AD utilizes a bidirectional recurrent convolutional network to further learn the correlations between long-term system behaviors inherent in the graph embedding sequence.

2.3. Deep learning-based methods

Learning-based methods leverage graph neural networks to extract node features from provenance graphs for detecting APT attacks. Existing learning-based methods can be divided into supervised and unsupervised approaches. In supervised detection, Prov-Gem (Kapoor et al., 2021) proposes a multi-embedding approach to capture semantic information of nodes, and jointly trains a GCN encoder and a classifier to report graph-level anomalies. APT-KGL (Chen et al., 2022) uses heterogeneous graph embedding techniques to generate node embeddings, and then detects node anomalies using a relational graph convolutional network (RGCN). ATLAS (Alsaheel et al., 2021) applies lemmatization and word embedding to extract sequences and uses an LSTM network for classification. However, these models require labeled attack data for training, which are often scarce and difficult to obtain. Additionally, they are not robust against zero-day vulnerabilities, as they depend on previously seen attack patterns.

For unsupervised detection, more recent work (Jia et al., 2023; Paudel & Huang, 2022; Rehman et al., 2024; Wang et al., 2022; Yang et al., 2023; Zengy et al., 2022) adopts anomaly-based methods, i.e., learning the normal behavior of the system and detecting activities that deviate from it. Threatrace (Wang et al., 2022) uses GraphSAGE to learn the behavior of benign system entities in the provenance graph and trains multiple models to detect anomalous nodes. Shadewatcher (Zengy et al., 2022) applies the recommendation concepts of user-item interactions to cyber-threat detection, detecting cyber-threats at the edge level by predicting system entities' preferences for the entities with which they interact. MAGIC (Jia et al., 2023) leverages masked graph representation learning to model benign entities, and then identifies anomalous system behaviors via outlier detection methods. However, the above approaches are static models that cannot handle dynamic graphs. They are unable to capture temporal information and dynamic behavioral patterns, which are also crucial for detecting threats. Some dynamic-learning methods, such as Pikachu (Paudel & Huang, 2022) and PROGRAPHER (Yang et al., 2023), decompose dynamic graphs into discrete static snapshots for separate representation learning. These methods may lead to graph structure fragmentation, hindering the continuous analysis of dynamic changes. Moreover, static snapshots represent the state of the graph within a specific time interval $[t - \delta, t]$, rather than encompassing all information from 0 to time t . Instead of operating with periodic graph snapshots, CGL-AD works with continuous-time dynamic provenance graphs, i.e., a stream of timestamped events, similar to Kairos (Cheng

et al., 2023). Kairos (Cheng et al., 2023) uses an encoder-decoder structure based on dynamic graph neural network to reconstruct edge types. It then utilizes the reconstruction error and rarity of nodes to achieve time window-level anomaly detection. Recent work, FLASH (Rehman et al., 2024), combines Word2Vec with positional encoding to capture semantic attributes and temporal information, then employs GraphSage to learn node embeddings and identify anomalies. But they perform embedding learning only from a global perspective, ignoring the local heterogeneous structural features around nodes. To address this gap, CGL-AD performs multi-perspective embedding learning. It integrates a frequency estimation technique with dynamic GNNs, thoroughly capturing the rich contextual information within the provenance graph.

3. Methodology

In this section, we first provide definitions for the provenance graph and the problem statement. Based on these definitions, we then introduce the proposed model. We present an overview of CGL-AD in Section 3.2, followed by the detailed design of each module in Sections 3.3 to 3.6.

3.1. Preliminary

System provenance graph. We define a provenance graph as $G = \{\mathcal{V}, \mathcal{E}, \mathcal{X}_{\mathcal{V}}, \mathcal{X}_{\mathcal{E}}, \mathcal{T}_{\mathcal{E}}\}$, where $\mathcal{V}$ is the set of vertices and $\mathcal{E}$ is the set of directed edges with timestamp attribute. $\mathcal{V}$ represent system entities (process, thread, file, socket, etc.), and $\mathcal{E} = \{(u, v) | u, v \in \mathcal{V}\}$ represents system call events (create, open, read, write, connect, etc.) between entities. The function $\mathcal{X}_{\mathcal{V}} : \mathcal{V} \rightarrow \cup_{\mathcal{V}}$ assigns a label to each node from node type set $\cup_{\mathcal{V}}$, and $\mathcal{X}_{\mathcal{E}} : \mathcal{E} \rightarrow \cup_{\mathcal{E}}$ assigns a label to each edge from edge type set $\cup_{\mathcal{E}}$. $\mathcal{T}_{\mathcal{E}}$ records the timestamp when each edge occurs.

Problem statement. We formalize the APT detection problem as a graph anomaly detection task on large-scale streaming provenance graphs. Since attackers tend to exploit zero-day vulnerabilities, we assume that we have no prior knowledge of any attack patterns. Our goal is to model normal behavior from benign data and detect anomaly in an unsupervised manner. We denote each graph as an evolving graph $G = \{G_{t_1}, G_{t_2}, \dots, G_{t_x}\}$. We train CGL-AD on a set of normal provenance graphs obtained from a benign system runtime environment. During testing, CGL-AD classifies a new provenance graph, G' as benign ($y = 0$) or malicious ($y = 1$) by evaluating its alignment with the system patterns learned from the training corpus.

3.2. Model overview

The system architecture of CGL-AD is illustrated in Fig. 2, which consists of the following four key modules:

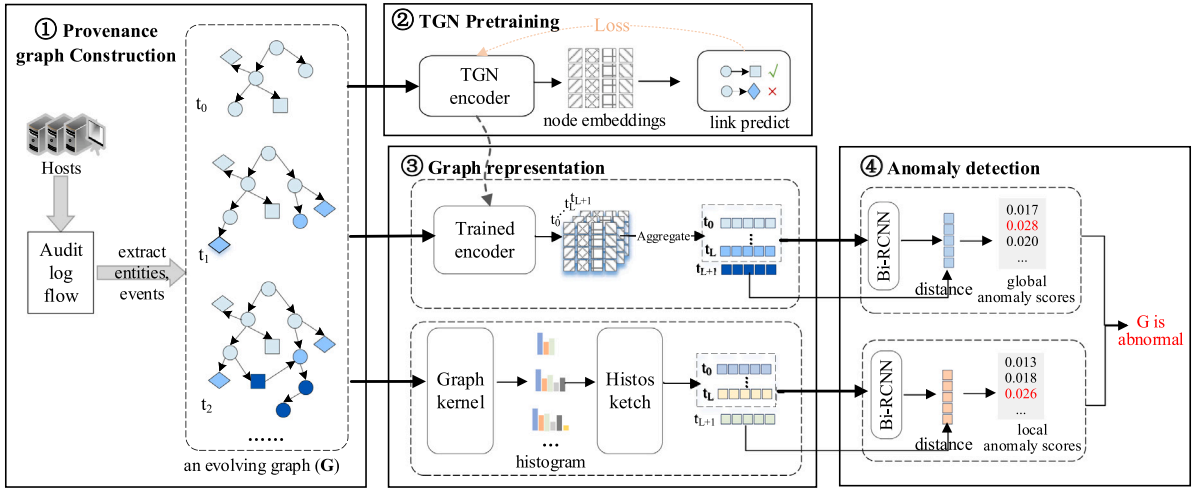


Fig. 2. Overview of CGL-AD architecture.

- ① **Provenance graph Construction** (Section 3.3). This component involves processing the stream of audit logs from t_0 to t_x collected by the OS audit tool. We extract entities and events from the logs and organize them into batches of time-ordered edges, which form a dynamic provenance graph G that evolves over time.
- ② **TGN pretraining** (Section 3.4). This component pretrains a temporal graph network (TGN) encoder through a link prediction task (Algorithm 1). Input G into the encoder in the form of batches of edge flows. In each batch, the node embeddings are updated, and the loss is calculated to update the TGN's parameters. The trained TGN encoder is then capable of generating high-quality semantic representations.
- ③ **Graph representation** (Section 3.5). Given batches of edge flows, CGL-AD uses the trained TGN to infer node embedding. After processing a specified number of edges (marked as different moments $t_0, t_1, \dots$ in Fig. 2), the embeddings of all encountered nodes are aggregated into a graph embedding, which represents the system's state at that moment. Simultaneously, CGL-AD applies a frequency estimation technique based on graph kernel and Histogram sketching to generate graph sketch vectors at various time points. Ultimately, CGL-AD obtains two distinct vector sequences for the same evolving graph, summarizing the system's operational state changes from different perspectives.
- ④ **Anomaly detection** (Section 3.6). Based on the two sequences of graph representations output by the previous module, CGL-AD trains two Bi-RCNN models as presented in Algorithm 2. Both Bi-RCNNs take a representation sequence of length L as input, with the goal of predicting the next adjacent representation. For a test graph G , the trained CGL-AD model produces two lists of anomaly scores: a global list S_1 and a local list S_2 , based on the sketches of G from time t_{L+1} to t_x . The graph G is deemed abnormal if $\text{Max}(S_1) > \theta_1$ and $\text{Max}(S_2) > \theta_2$, with θ_1 and θ_2 being thresholds set through cross-validation.

3.3. Provenance graph construction

Similar to many other approaches (Han et al., 2020; Milajerdi, Gjomemo et al., 2019), we use Camflow (Pasquier et al., 2017), a kernel-level logging framework, to construct a whole-system, time-ordered provenance graph. Camflow captures detailed information about system activities at runtime, including inter-process communication, file operations, network connections, and more. This data is then transformed into a machine-readable format, typically JSON. For illustrative purposes, an simplified example of such an event is shown in Fig. 3. This event can be directly translated into an edge within

```
{
  "type": "WasGeneratedBy",
  "from": "ABACi2w",
  "to": "AQECk2x",
  "annotations": {
    "id": "IAQD0Cw",
    "relation_id": "3060",
    "relation_type": "write",
    "cf:machine_id": "cf:1164181",
    "cf:date": "2020-07-11T10:41:28",
    "jiffies": "4294759940",
    "task_id": "56228",
    "from_type": "file",
    "to_type": "process"
  }
}
```

Fig. 3. An example of write event recorded by Camflow.

the provenance graph. In the example, the edge represents a “write” operation from process AQECk2x to file ABACi2w, with a timestamp of 2020-07-11T10:41:28.

Given a stream of logs collected by the auditing tool (i.e., Camflow) from hosts, CGL-AD extracts the necessary fields from the json logs, parses them into triplets of $(src, \mathcal{X}_{src}, dst, \mathcal{X}_{dst}, \mathcal{X}_{operation}, t)$, and inserts them as directed edges into the provenance graph. Unlike most existing methods, CGL-AD analyzes the provenance graph as a continuous-time dynamic graph rather than a static graph. The provenance graph is enriched with contextual information that describes the continuous transformation of system behavior, including the structural relationships between entities and the temporal relationships of events. This enables CGL-AD to comprehensively model normal system behavior and effectively identify anomalies.

3.4. TGN pretraining

While some researchers advocate for employing graph learning methods to capture the structural information of provenance graphs (Jia et al., 2023; Yang et al., 2023), these approaches often focus on static snapshots, neglecting the crucial temporal dynamics. To address this limitation, CGL-AD uses temporal graph network (TGN) (Rossi et al., 2020) to encode both the structure and temporal information in provenance graphs. TGN dynamically tracks node evolution through

time encoding and message mechanism. It includes a memory module that stores the latest state vector for each node, and leverages accumulated historical information and multi-hop message propagation to capture the global structure of the graph.

Initially, the state of newly appearing nodes is initialized to a vector with all zeros. When a new edge $e_{uv}(t)$ arrives, messages are computed for both the source node u and the destination node v :

$$\begin{aligned} m_u(t) &= s_u(t^-) \parallel s_v(t^-) \parallel \text{mlp}(\Delta t_u) \parallel e_{uv}(t), \\ m_v(t) &= s_v(t^-) \parallel s_u(t^-) \parallel \text{mlp}(\Delta t_v) \parallel e_{uv}(t) \end{aligned} \quad (1)$$

where $s_u(t^-)$ denotes the memory of node u before time t , Δt_u is the time interval between t and the last interaction of node u , $e_{uv}(t)$ represents the edge. Each edge is encoded as a concatenation of source node's feature, destination node's feature and a one-hot vector of the edge type. The initial feature of a node is the one-hot vector of its type. If multiple edges in a batch involve the same node i , the messages $m_i(t_1), \dots, m_i(t_b)$ are aggregated into a single message for $t_1, \dots, t_b \leq t$:

$$\bar{m}_i(t) = \text{agg}(m_i(t_1), \dots, m_i(t_b)) \quad (2)$$

Then the new memory representation of node i is updated by a Gated Recurrent Unit (GRU) (Cho et al., 2014):

$$s_i(t) = \text{GRU}(\bar{m}_i(t), s_i(t^-)) \quad (3)$$

Subsequently, for each node i , CGL-AD aggregates information from its temporal neighborhood $\mathcal{N}$ and generates the final embedding $z_i(t)$ using a transformer-based Graph Attention Network (GAT) (Shi et al., 2020):

$$z_i(t) = \text{GAT}(s_i(t), \mathcal{N}, t) \quad (4)$$

where t is a vector of timestamps, $\mathcal{N}$ includes the memory information of i 's neighboring nodes. The graph attention is able to select which neighbors are more important based on both features and timing information.

For our experiments, we randomly sampled several graphs from benign graphs and optimized the model for link prediction to pretrain the TGN encoder. The optimization objective is to minimize loss

$$\mathcal{L} = -\log(\sigma(z_u^T z_v)) - \log(1 - \sigma(z_u^T z_{v'})) \quad (5)$$

where $\{(u, v) \in \tilde{\mathcal{E}}\}$, $\tilde{\mathcal{E}} \in \mathcal{E}$ is the set of edges that exist in the sampled graphs, and $\{(u, v') \notin \mathcal{E}\}$ are randomly sampled negative pair from non-edges. Note that when processing batches during training, we use the messages from previous batches to update the memory and predict interactions in current batch. This prevents the issue of information leakage. Algorithm 1 presents the procedure of TGN pretraining. After the pretraining is completed, the parameters of TGN will be frozen and no fine-tuning will be conducted subsequently.

Algorithm 1 Pretraining

Input: Training provenance graphs $G_{\text{pretrain}} = \{G_1, \dots, G_{n_1}\}$, where $G_i = \{\mathcal{V}, \mathcal{E}, \mathcal{X}_v, \mathcal{X}_e, \mathcal{T}_e\}$, number of epochs K_1

Output: Trained TGN Encoder

```

1: for epoch = 1 to  $K_1$  do
2:   for  $G_i$  in  $G_{\text{pretrain}}$  do
3:      $G'_i \leftarrow \text{TGN}(G_i) \triangleright$  update node embeddings
4:     for all  $(u, v) \in \mathcal{E}$  do
5:       Sample negative pairs  $(u, v') \notin \mathcal{E}$ 
6:       Compute the loss  $\mathcal{L}$  as Eq. 5
7:     end for
8:     Update parameters of the TGN Encoder
9:   end for
10: end for

```

3.5. Graph representation

Note that the provenance graph is characterized by heterogeneous types of nodes and edges, where node types include file, process, etc., and edge types include various system calls such as read, write, fork, and other parent-child relations. Given the complexity and diversity of system calls within the graph, it is essential to learn the relationships between system entities from multiple levels. However, existing methods often overlook the critical complementary relationship between the global graph and local subgraph contexts, making it difficult to distinguish subtle differences between normal and malicious activities, which leads to false positives or missed attacks. CGL-AD bridges this gap by combining both global graph and local heterogeneous subgraph perspectives to fully capture the rich information contained in the provenance graph.

Global representation. On the one hand, CGL-AD uses the pre-trained TGN encoder (Section 3.3) to capture global contextual information. For each time point t , we denote the snapshot of G at that moment as G_t . The embedding of the observed nodes are represented as

$$\{z_1(t), z_2(t), \dots, z_n(t)\} \quad (6)$$

where n is the number of nodes in G_t , and each embedding $z_i(t)$ contains historical change information up to time t . We aggregate these node embeddings to generate a global representation of the graph at time t . Here, we use a simple averaging operation as below

$$Z_{G_t}^{\text{global}} = \sum_{k=1}^n z_k(t)/n \quad (7)$$

Local representation. On the other hand, inspired by previous work (Han et al., 2020), we use a hash-based frequency estimation technique to capture the local heterogeneous context. This technique preserves important local substructure information of the graph in a compact sketch vector, which contains of two steps: (i) calculating the frequency of local heterogeneous substructures, (ii) generating graph sketch vectors.

Specifically, we adapt a linear-time, fast Weisfeiler-Lehman (WL) subtree graph kernel algorithm (Shervashidze, Schweitzer, Van Leeuwen, Mehlhorn, & Borgwardt, 2011) to capture the forward R-hop neighborhood structure of each nodes, which we define as the subgraph marker. The subgraph marker is a string constructed through an R-hop breadth-first traversal (BFS) as follows: First, initialize the subgraph marker with the node type: $l(v, R) = \mathcal{X}_v$. Then, traverse the incoming edges of node v in timestamp order. For each traversed edge e , concatenate the type of the source node u and the edge e with the subgraph marker: $l(v, R) = \mathcal{X}_v \parallel \mathcal{X}_e \parallel l(v, R)$. Then, we build a frequency histogram H . Each element in the histogram represents the occurrence count of a unique subgraph marker.

Based on the insight that a normal evolving graph should exhibit high similarity at neighboring time points, we compare the states of graph at adjacent time points by the relative frequencies of substructures. To achieve this, we use Histosketch (Yang, Li, Rettig, & Cudré-Mauroux, 2017), a locally sensitive hashing (LSH) method, to map the histogram to a numerical vector. To generate the vector of size ϕ , we first sample the following random variables: $\gamma_{i,j} \sim \text{Gamma}(2, 1)$, $\lambda_{i,j} \sim \text{Gamma}(2, 1)$, $\beta_{i,j} \sim \text{Uniform}(0, 1)$ for $j = 1, 2, \dots, \phi$ and $i \in \mathcal{E}$. For each time t , Histosketch is used to summarize the histogram at that moment into a sketch vector $Z_{G_t}^{\text{local}}$. We compute each element j in the sketch vector as follows:

$$\begin{aligned} c_{i,j} &= \exp(\log H_i + \gamma_{i,j} \beta_{i,j}) \\ a_{i,j} &= \lambda_{i,j} / (c_{i,j} \exp(\gamma_{i,j})) \\ Z_{G_t}^{\text{local}}(j) &= \argmin_{i \in \mathcal{E}} a_{i,j} \end{aligned} \quad (8)$$

where ε is the set of subgraph markers, and H_i is frequency value of item i . This algorithm effectively maintain fixed-size sketch vectors for streaming histograms, providing an unbiased approximation of the similarity between histograms. We refer interested readers to the original work (Yang et al., 2017) to prove the correctness of the algorithm.

3.6. Anomaly detection

Since APT scenarios typically persist over extended periods, continuously monitoring the state changes of the provenance graph is crucial for threat detection. However, most previous methods overlook this aspect (Han et al., 2020; Jia et al., 2023; Manzoor et al., 2016). These methods focus on extracting embeddings for clustering and outlier analysis, failing to capture the system's evolutionary behavior, which leads to high false positives. To address this limitation, We use a sequence learning module to capture the normal changes in system behavior. We adopt the bidirectional recurrent convolutional neural network (Bi-RCNN) proposed by Lai, Xu, Liu, and Zhao (2015), which has been widely used for text classification task.

CGL-AD trains two Bi-RCNNs to learn patterns from the global and local representation sequences output by the previous module, respectively. Since the process is identical for both, we use a wildcard “*” in the following text to represent either global or local perspective. The Bi-RCNN takes a sequence of sketch embeddings $\{Z_{G_{t_1}}^*, Z_{G_{t_2}}^*, \dots, Z_{G_{t_L}}^*\}$ as input, with the goal of predicting the next sketch $Z_{G_{t_{L+1}}}^*$. We firstly define $C_l^*(G_{t_i})$ and $C_r^*(G_{t_i})$ as left and right context embedding of G_{t_i} respectively:

$$\begin{aligned} C_l^*(G_{t_i}) &= Relu(W^{(l)}C_l^*(G_{t_{i-1}}) + W^{(sl)}Z_{G_{t_{i-1}}}^*) \\ C_r^*(G_{t_i}) &= Relu(W^{(r)}C_r^*(G_{t_{i+1}}) + W^{(sr)}Z_{G_{t_{i+1}}}^*) \end{aligned} \quad (9)$$

where $Relu$ is the activation function and $W^{(l)}$, $W^{(sl)}$, $W^{(r)}$, $W^{(sr)}$ are weight matrices. Then we concatenate the left-side context vector, the sketch embedding, and the right-side context vector to obtain $\eta_i^* = [C_l^*(G_{t_i}); Z_{G_{t_i}}^*; C_r^*(G_{t_i})]$ as representation of sketch G_{t_i} , which captures the semantics of both its left- and right-side contexts.

The recurrent and convolutional network are used to obtain a latent representation e_i^* of each sketch G_{t_i} in the input sequence. When all of the representations of sketches are calculated, we apply a max-pooling layer followed by a fully-connected layer to obtain the predicted representation $\hat{Z}_{G_{t_{L+1}}}^*$, defined as:

$$\begin{aligned} e_i^* &= tanh(W^{(1)}\eta_i^* + b^{(1)}) \\ \hat{Z}_{G_{t_{L+1}}}^* &= (W^{(2)}\max_{i=1}^L(e_i^*) + b^{(2)}) \end{aligned} \quad (10)$$

where W represents the weight matrices, b is the bias vector, and $\max$ denotes an element-wise pooling operation.

We define the loss function as the L2 distance between the predicted representation and the ground-truth representation, which serves as the optimization objective for training:

$$\mathcal{L}_p^* = \|Z_{G_{t_{L+1}}}^* - \hat{Z}_{G_{t_{L+1}}}^*\|_2 \quad (11)$$

The training procedure is summarized in Algorithm 2. Since APT attacks are typically multi-staged and covert, the activities in each stage may have different impacts on the global and local graph structures. This means that local and global anomalies might manifest at different times. CGL-AD trains two Bi-RCNNs to separately learn the pattern changes in the global structure and local subgraphs.

Algorithm 2 Training

Input: Training provenance graphs $G_{train} = \{G_1, \dots, G_{n_2}\}$, interval of sketch generation bs , number of epochs K_2 , hop count R , sequence length L

Output: Trained Bi-RCNN models $rcnn_1$ and $rcnn_2$

```

1: for epoch = 1 to  $K_2$  do
2:   for  $G_i$  in  $G_{train}$  do
3:      $x \leftarrow \lfloor |G_i|/bs \rfloor \triangleright |G_i|$  is number of edges in  $G_i$ 
4:      $\{G_i(t_0), \dots, G_i(t_x)\} \leftarrow \text{split}(G_i, bs)$ 
5:      $Z_{G_i}^{global} \leftarrow [], Z_{G_i}^{local} \leftarrow []$ 
6:     for  $j = 1$  to  $x$  do
7:        $Z_{t_j}^{global} \leftarrow \text{Mean}(TGN(G_i(t_j)))$ 
8:        $Z_{t_j}^{local} \leftarrow \text{histosketch}(\text{graphkernel}(G_i(t_j), R)$ 
9:       Append  $Z_{t_j}^{global}$  to  $Z_{G_i}^{global}$ 
10:      Append  $Z_{t_j}^{local}$  to  $Z_{G_i}^{local}$ 
11:     end for
12:      $\mathcal{L}_p^{global} = 0, \mathcal{L}_d^{local} = 0$ 
13:     for  $j = L$  to  $x - 1$  do
14:        $\hat{Z}_{G_{t_{j+1}}}^{global} \leftarrow rcnn_1(Z_{G_{t_j-L}}^{global}, \dots, Z_{G_{t_j}}^{global})$ 
15:        $\hat{Z}_{G_{t_{j+1}}}^{local} \leftarrow rcnn_1(Z_{G_{t_j-L}}^{local}, \dots, Z_{G_{t_j}}^{local})$ 
16:        $\mathcal{L}_p^{global} + = \| \hat{Z}_{G_{t_{j+1}}}^{global} - Z_{G_{t_{j+1}}}^{global} \|_2$ 
17:        $\mathcal{L}_p^{local} + = \| \hat{Z}_{G_{t_{j+1}}}^{local} - Z_{G_{t_{j+1}}}^{local} \|_2$ 
18:     end for
19:     Update parameters of  $rcnn_1$  and  $rcnn_2$ 
20:   end for
21: end for

```

During the testing phase, CGL-AD integrates both global and local perspectives to capture anomalies in the global and local structures at different times. The prediction deviation is computed as the anomaly score. For a test graph G , the trained CGL-AD model produces two lists of anomaly scores: a global list S_1 and a local list S_2 , based on the sketches of G from time t_{L+1} to t_x . Each anomaly score represents the distance between the predicted and ground-truth representations. The graph G is considered abnormal if $\text{Max}(S_1) > \theta_1$ and $\text{Max}(S_2) > \theta_2$, where θ_1 and θ_2 are thresholds set through cross-validation.

4. Experimental results and discussion

Our evaluation focuses on answering the following research questions:

Q1. How effective is CGL-AD as an APT attack detection system? (Section 4.2)

Q2. How do hyperparameters affect CGL-AD' detection performance? (Section 4.3)

Q3. How important is the design of each modules to facilitate detection? (Sections 4.4, 4.6)

Q4. How efficient is CGL-AD? (Section 4.5)

Guided by the aforementioned questions, we designed and conducted a series of experiments using three datasets. This section firstly provides an overview of the datasets, baselines, evaluation metrics, and implementation details of the experiment. Then, we perform comprehensive experiments to address the posed questions.

4.1. Experimental setup

4.1.1. Datasets

We conducted experiments using three public available datasets, which are frequently used by state-of-the-art provenance-based threat detectors.

Streamspot dataset. The statistics of the graphs in Streamspot dataset is shown in Table 2. StreamSpot is a simulated dataset collected and

Table 2
Statistics of the Streamspot dataset.

Dataset	Scenarios	# of Graphs	Avg # of nodes	Avg # of edges
Streamspot	YouTube	100	8292	113,229
	Gmail	100	6827	37,382
	Video Game	100	8831	310,814
	Attack	100	8891	28,423
	Download	100	8637	112,958
	CNN	100	8990	294,903

Table 3
Statistics of the Camflow-apt and the Shellshock datasets.

Dataset	Scene	# of graph	Avg # of nodes	Avg # of edges
Camflow-apt	Benign	125	265,424	975,226
	Attack	25	257,156	957,968
Shellshock	Benign	125	238,338	911,153
	Attack	25	243,658	949,887

publicly released by StreamSpot (Manzoor et al., 2016), using the auditing system SystemTap (Prasad et al., 2005). It contains 600 information flow graphs derived from five benign scenarios and one attack scenario. The benign scenarios cover a variety of activities, including checking Gmail, browsing the CNN.com website, downloading files, watching YouTube, and playing video games. The attack scenario simulates a drive-by-download attack, involving malicious activities driven by a malicious URL, exploiting Flash vulnerabilities to gain root access to the host.

Camflow-apt dataset & Shellshock dataset. Table 3 summarizes the characteristics of these datasets. The Camflow-apt and Shellshock datasets are collected by Unicorn (Han et al., 2020) in a controlled lab environment following the typical cyber kill chain model. They each include 150 graphs, with each graph representing the whole-system provenance of a host running for three days. There are background benign activities in both benign and attack graphs. The attack scenario simulate supply-chain attacks designed to subvert a company's software distribution channel to spread malware. In the Camflow-apt dataset, the attacker exploited vulnerability from GNU wget version 1.17 (CVE-2016-4971) to embed a common remote access trojan (RAT) into a Debian package. When the CI server downloaded and installed the package, RAT established a C&C channel, allowing attacker to control the CI server and modify configuration. In the Shellshock dataset, the attacker exploited a different vulnerability in GNU Bash version 4.3 (CVE-2014-6271) to execute arbitrary commands on a vulnerable server by injecting malicious code into environment variables in bash scripts.

In our experiments, all data were preprocessed into formatted CSV files as described in Section 3.2. Each line has the following format: source_id, source_type, destination_id, destination_type, edge_type, timestamp. The benign and attack graphs are distinguished by graph ID (i.e., file name).

4.1.2. Baselines

In order to evaluate the performance of our model, we compare CGL-AD with six baselines of provenance-based APT detectors on graph classification task. These include two statistic-based methods (Streamspot Manzoor et al., 2016, Unicorn Han et al., 2020), and four deep learning-based methods, among which are two static models (Threatrace Wang et al., 2022, MAGIC Jia et al., 2023) and two dynamic models (Kairos Cheng et al., 2023, FLASH Rehman et al., 2024). The details of these baselines are summarized as follows:

- **Streamspot** (Manzoor et al., 2016) decomposes each graph into a series of k-shingles and constructs their frequency vectors. It then uses SimHash to map the shingle count vectors to fixed low-dimensional vectors and applies k-medoids for graph-level anomaly detection.
- **Unicorn** (Han et al., 2020) uses the data stream frequency estimate technique to obtain sketch vector. Similar to Streamspot, they both perform graph-level embedding and use k-medoids to identify anomalies.
- **Karios** (Cheng et al., 2023) utilizes a temporal graph networks-based encoder-decoder architecture to quantify the abnormality of each edge through reconstruction errors of edge types, then combines the rarity of nodes to detect time window-level anomalies.
- **Threatrace** (Wang et al., 2022) uses GraphSAGE to learn the behavior of benign system entities and trains multiple models to detect anomalous nodes based on deviations from learned behavior.
- **MAGIC** (Jia et al., 2023) leverages masked graph representation learning to model benign entities, then identifies anomalous system behaviors via outlier detection methods.
- **FLASH** (Rehman et al., 2024) uses an improved word2vec to encode semantic attributes of nodes, employs GraphSAGE to learn node embeddings, then uses XGBoost to identify anomalous nodes.

For the node-level models Threatrace and FLASH, following the approach used in their original work, we transformed node-level detection into graph-level detection for our evaluation datasets that lack node-level ground truth label. Specifically, if the number of anomalous nodes exceeds a predefined threshold, the graph is considered anomalous.

4.1.3. Evaluation metrics

Following previous works (Han et al., 2020; Wang et al., 2022), when an attack graph is correctly detected, it is counted as a true positive (TP); when it is not detected, it is counted as a false negative (FN). True negatives (TN) and false positives (FP) are defined for benign graphs accordingly. We then compute the following metrics: accuracy, precision, recall, F1 score, false positive rate (FPR), AUC and equal error rate (EER). AUC refers to the area under the ROC curve; EER represents the point where the false positive rate equals the false negative rate, offering a balanced assessment of errors. Higher AUC and lower EER indicate better performance. These metrics are chosen to provide a comprehensive evaluation of the model's performance. Additionally, we conduct the Wilcoxon signed-rank test (Gibbons & Chakraborti, 2014) to assess the statistical significance of the difference between CGL-AD and the other methods.

$$\begin{aligned}
 accuracy &= \frac{TP + TN}{TP + TN + FP + FN} \\
 precision &= \frac{TP}{TP + FP}, \quad recall = \frac{TP}{TP + FN} \\
 F1 &= 2 \times \frac{precision \times recall}{precision + recall}, \quad FPR = \frac{FP}{FP + TN}
 \end{aligned} \tag{12}$$

4.1.4. Implementation details

For all baseline methods, we use the source code provided in their original papers and follow the parameter settings introduced by authors. Our proposed model, CGL-AD, is implemented using Python and C++. We implement the data parsing, the TGN encoder and the anomaly detection model in Python 3.7 with about 2000 lines of code. For the frequency estimation module, we refer to the C++ implementation from Han et al. (2020). The entire training process is divided into two stages. In the first stage, we pretrain the TGN encoder using a small subset of graphs from the training set (five in our experiments), and validate it on a small portion of graphs from the validation set. The second stage involves training the anomaly detector,

Table 4

Experiment results for CGL-AD and baseline methods on three datasets. The best result is highlighted in bold. “p-value” represents the result of the Wilcoxon signed-rank test (with $\alpha = 0.05$). The p -value < 0.05 indicates that the result is statistically significant. The p -value > 0.05 is marked in underline.

Dataset	Method	Accuracy (%/p-value)	Precision (%/p-value)	Recall (%/p-value)	F1 (%/p-value)	AUC (%/p-value)	FPR	EER
Streamspot	Unicorn	93.47/1.6e-5	88.63/1.6e-5	99.73/3.8e-3	93.85/1.6e-5	92.52/1.2e-5	0.1279	0.1082
	Streamspot	86.99/4.1e-5	84.26/3.2e-5	91.00/2.7e-5	87.49/4.1e-5	89.74/2.1e-5	0.1699	0.1250
	Threatrace	92.20/1.4e-5	89.08/1.2e-5	96.20/1.2e-5	92.49/2.6e-5	91.88/1.4e-5	0.1180	0.1197
	MAGIC	98.59/4.6e-4	98.28/9.5e-4	98.93/ <u>5.7e-2</u>	98.60/9.5e-4	97.74/2.6e-4	0.0173	0.0080
	FLASH	97.56/2.3e-5	98.97/6.1e-5	96.13/1.9e-5	97.53/3.0e-5	97.56/6.1e-5	0.0102	0.0084
	CGL-AD(ours)	99.07/-	99.07/-	99.07/-	99.07/-	99.20/-	0.0093	0.0064
Camflow-apt	Unicorn	95.07/1.5e-3	93.15/6.1e-5	97.33/ <u>5.6e-2</u>	95.17/8.5e-4	92.63/1.5e-3	0.0720	0.0615
	Threatrace	94.79/1.8e-3	93.09/9.3e-4	96.80/ <u>7.6e-2</u>	94.89/1.8e-3	92.21/1.6e-3	0.0720	0.0720
	Kairos	72.53/1.3e-4	70.87/1.7e-5	76.53/3.8e-4	73.59/1.2e-4	76.35/1.2e-4	0.3146	0.2900
	MAGIC	89.47/3.5e-5	83.49/6.2e-4	98.39/2.2e-2	90.32/7.1e-4	90.47/2.1e-5	0.1946	0.1600
	FLASH	94.66/5.7e-4	92.53/3.3e-4	96.00/2.7e-3	94.74/1.6e-4	93.14/1.4e-4	0.0667	0.0667
	CGL-AD(ours)	97.47/-	98.15/-	96.80/-	97.45/-	96.83/-	0.0187	0.0150
Shellshock	Unicorn	80.93/1.3e-4	76.48/2.1e-4	89.33/7.5e-4	82.39/2.6e-5	84.58/1.0e-4	0.2746	0.2250
	Threatrace	86.27/1.0e-4	80.22/1.7e-5	96.27/4.2e-2	87.50/1.3e-4	86.59/1.3e-4	0.2373	0.2187
	Kairos	76.13/1.2e-4	73.89/3.1e-5	81.06/1.7e-4	77.27/1.2e-4	79.72/1.4e-4	0.2880	0.2610
	MAGIC	82.40/2.7e-4	77.33/1.1e-4	91.73/1.3e-2	83.91/3.8e-3	81.94/2.1e-4	0.2640	0.2480
	FLASH	91.86/7.8e-4	89.26/1.6e-4	95.20/3.1e-4	91.26/1.2e-4	88.04/1.2e-4	0.1146	0.1020
	CGL-AD(ours)	94.27/-	98.59/-	90.87/-	94.10/-	93.63/-	0.0233	0.0182

where the pretrained TGN and frequency estimation technique are employed to generate graph embedding sequences. These sequences are then input into the Bi-RCNN for training. The encoder and anomaly detector are optimized using the Adam optimizer. To mitigate overfitting, we employ a dropout technique. Additionally, we further use an early stopping strategy with a tolerance of 30 epochs, the training will be terminated if the accuracy on the validation set does not increase for 5 consecutive epochs. Hyper-parameters are chosen through grid search, and their optimal values are described below. In Section 4.3, we discuss the impact of different parameters.

- **TGN encoder:** The batch size for processing edge streams is 1000; the learning rate is $5e-5$, the pretrain epoch is 10, and the dropout rate is 0.1.
- **Frequency estimation:** we follow the setup in Han et al. (2020). The hop count R for generating subgraph markers is 3; the dimension of local representation is 2000; the weight decay factor λ is 0.02.
- **Anomaly detector:** The dimension of the hidden layer is 128; the number of the hidden layer is 2; the initial learning rate is $1e-5$; the dropout rate is 0.2.

In each experiment, we follow the settings of prior works (Han et al., 2020; Wang et al., 2022) and perform 4-fold validation on all datasets. Specifically, for the Streamspot dataset, in each fold, we select 25 graphs from each benign scenario for testing, along with all attack graphs (a total of 225 graphs). 5 graphs from each benign scenario are held out for validation (a total of 25 benign graphs), and the remaining 350 benign graphs are used as the training set. For the Camflow-apt and Shellshock dataset, in each fold, 25 benign graphs and all attack graphs form the test set, with 15 benign graphs reserved for validation and the remaining 85 benign graphs as the training set.

All experiments are performed on a server with Intel (R) Core (TM)2 Duo CPUs T7700 @ 2.40 GHz, 128 GB of physical memory, and an NVIDIA Tesla T4 GPU. The operating system is Ubuntu 18.04.6 LTS.

4.2. Performance comparison

To demonstrate the effectiveness of CGL-AD model, we present the performance comparison of CGL-AD and other baseline methods in graph-level attack detection. Table 4 shows the average metric results from three runs. Overall, CGL-AD achieves the best detection

performance across all datasets, and the p -value indicates that the improvement is statistically significant.

From Table 4, we observe that Kairos performs the worst. Although Kairos is a method based on dynamic graph embedding learning, it uses the rarity of nodes as a key criterion in the anomaly detection process. However, rarity itself is not always a reliable indicator of anomalies. Kairos’ reliance on rarity may lead to two issues: on the one hand, it might overlook frequent but subtle attack behaviors, such as attacks that gradually infiltrate the system through long-term, small-scale activities are not uncommon. On the other hand, it could easily misclassify rare but normal activities as anomalies.

Unicorn performs the best among the statistic-based baseline methods. It outperforms static learning methods on the Camflow-APT dataset due to its streaming capability, which captures temporal information in the graph. However, it treats sketch vectors as isolated data points and uses clustering to detect anomalies. This traditional distance-based model is highly sensitive to noise in the data, which explains why Unicorn has a high false positive rate.

FLASH performs the best among the dynamic learning-based baseline methods and outperforms the best static model, Threatrace. This is because Threatrace treats the graph as a monolithic structure, ignoring dynamic temporal information. For example, the order of abrupt edge appearances is crucial in distinguishing between regular downloads and malicious drive-by downloads. Additionally, FLASH outperforms Unicorn on the Streamspot and Shellshock datasets. The use of graph representation learning allows FLASH to gain deeper insights into benign behaviors compared to traditional models. However, FLASH overlooks the local heterogeneous structural features. In comparison, CGL-AD is 2.4% and 3.6% higher than FLASH on average in terms of F1 and AUC, respectively, and also significantly reduces the false positive rate by an average of 4.7%. We attribute this improvement to CGL-AD’s multi-perspective embedding learning and sequential learning, which enables CGL-AD to capture rich contextual information in provenance graphs, including temporal dynamics, global structural information, and statistical feature of local substructures. For example, if a normal process does not show anomalies from a local perspective but exhibits anomalous access patterns from a global perspective, such behavior can be further validated and potentially ignored by CGL-AD, thus reducing false positives.

4.3. Hyper-parameter analysis

In this section, we analyze the importance of CGL-AD’s key parameters across all dataset. We use the same setup as in Section 4.1.4 for the

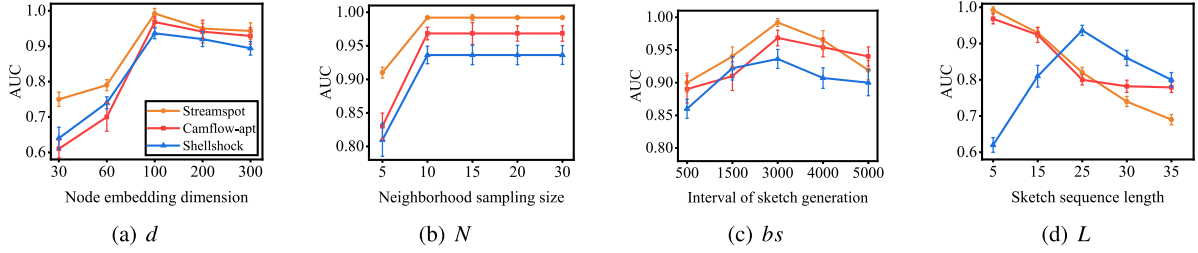


Fig. 4. AUC over all datasets w.r.t different hyper-parameters, the error bar indicates the standard error.

Table 5

Ablation study results on Camflow-apt.

Method	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
remove TGN	94.00	89.28	99.00	94.34
remove FET	84.00	77.42	96.00	85.72
concat embedding	80.02	82.61	76.00	79.98
TGN+K-means	73.28	69.06	78.16	74.51
CGL-AD(ours)	97.47	98.15	96.80	97.45

baseline configurations. We then vary each parameter independently to examine its impact. When investigating each hyper-parameter, we set the rest parameters to their optimal values found by grid search, and report the AUC performance on the test set. Fig. 4 shows the experimental results, with the error bars indicating the standard error.

Node embedding dimension (d). The node embedding z represents a node's state vector that captures the temporal evolution of the node neighborhood over time. As shown in Fig. 4(a), when the dimension d is too small, it is difficult for CGL-AD to retain information about past events. On the other hand, a large dimension may degrade the detection performance, as the state may contain outdated historical records that are irrelevant to the current event. The results show that CGL-AD can achieve the best performance when $d = 100$.

Neighborhood Sampling Size (N). During the process of learning node embeddings, if the number of sampled neighbors is too small, the embeddings may lack sufficient contextual information. However, as we continue to increase N , the detection performance no longer improves (Fig. 4(b)). We find $N = 10$ to be ideal among all datasets. This is because most of the nodes in the dataset have fewer than 10 neighboring nodes.

Interval of Sketch Generation (bs). This parameter determines the number of edges added to the graph between the construction of new sketches. It directly impacts the temporal granularity of the resulting sketch sequences. A smaller bs makes neighboring sketches too similar to each other, resulting in lower recall and accuracy. Conversely, a larger bs result in coarser granularity, which also makes the graphs look too similar overall. The duration of the anomaly pattern affects the choice of bs . We obtain optimal results when setting the interval to be around 3000 (Fig. 4(c)).

Sketch sequence length (L). For CGL-AD, the selection of the L is very important, because it directly affects the model's ability to capture anomalous patterns in the sequence. A small L may not provide enough contextual information, making it difficult for the model to understand long-term dependencies in the sequence. Some anomalous patterns may require longer sequences to be detected correctly. However, a large L may lead to underfitting and also increase computational and storage overheads. For Streamspot and Camflow-apt, the best result are achieved when $L = 5$, while for Shellshock, AUC achieves the best performance when $L = 25$. (Fig. 4(d)).

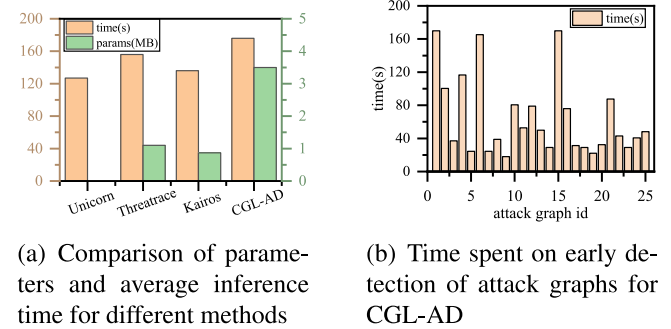


Fig. 5. Efficiency analysis on the Shellshock dataset. In (a), Unicorn is a non-machine learning method, where specific parameterization may not be applicable. In (b), ids 1, 6, and 15 are the attack graphs that were not correctly detected by CGL-AD (i.e., FNs).

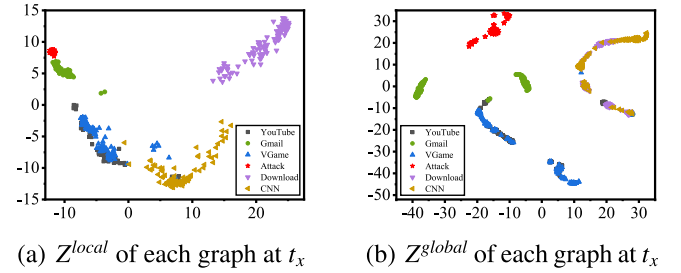


Fig. 6. Latent space of Streamspot dataset.

4.4. Ablation study

To investigate how our proposed framework facilitates APT detection, we conduct an ablation study to evaluate the contribution of different components of CGL-AD (Table 5). By removing the TGN module or the frequency estimation technique (FET) module, we observe a noticeable degradation in detection performance. This indicates that both the temporal graph learning module and frequency estimation embedding module have distinct sensitivities to anomalous patterns in dynamic provenance graphs. The “concat embedding” technique involves amalgamating global representations and local representations obtained from both embedding modules, and feeding them into a single Bi-RCNN for detection. The detection result demonstrates the performance improvement achieved by employing two Bi-RCNNs. We also replaced the Bi-RCNN model with a simple K-means method. The comparison of the “TGN+K-means” and “remove FET” result highlights that the Bi-RCNN effectively captures the inherent relationships and evolutionary patterns in the data, enabling CGL-AD to detect stealthy attacks with long latency periods. This is further supported by the visual analysis in Section 4.6.

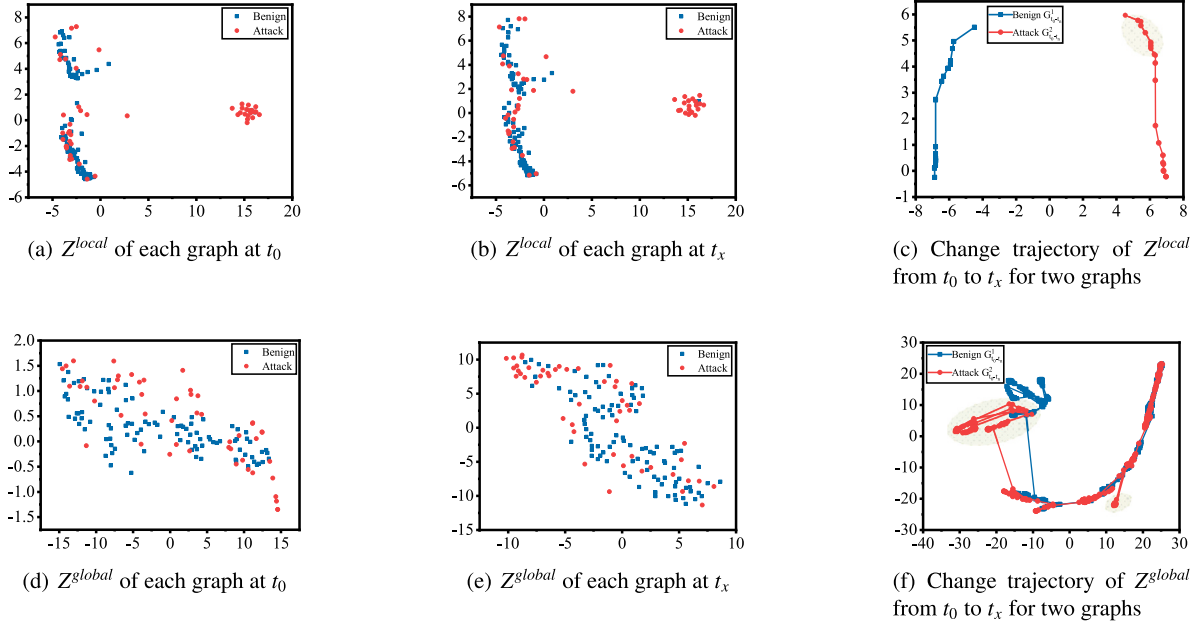


Fig. 7. Local and global representation space of Camflow-apt dataset, where t_0 refers to the time when only 10% of the edges of G are observed, t_x refers to the time when the entire graph G is observed. Shaded blocks in (c) and (f) indicate moments with a discernible change in trend distinct from the normal mode.

4.5. Efficiency analysis

We measure the number of model parameters and the average inference time of one graph in the Shellshock dataset, with the results displayed in Fig. 5(a). We observe that the CGL-AD algorithm demonstrates comparable numbers of parameters and running times to the baselines. However, it stands out with the highest accuracy and the lowest false positive rate in attack detection (as shown in Section 4.2). We believe this is an acceptable trade-off.

Additionally, it is worth noting that CGL-AD can detect early signs of APT attacks and raise alerts. This is because CGL-AD continuously monitors the state changes in the provenance graph and identifies anomalies through the deviation distance in sequence prediction. Specifically, it uses the graph sequence of $[t - L, t]$ to predict the graph state at time $t + 1$. A significant deviation indicates a potential attack occurring at time $t + 1$. Fig. 5(b) shows the time taken by CGL-AD to detect the first anomalous sketch when inferring the 25 attack graphs in the Shellshock dataset. Each graph contains an average of 268 sketches in the sequence. The attack graphs with IDs 1, 6, and 15 are false negative graphs (i.e., FNs), where CGL-AD fails to detect the anomaly after processing all edges. However, in the correctly detected attack graphs (i.e., TPs), CGL-AD can identify attack signs at an earlier stage, with an average time consumption of 49 s. Therefore, CGL-AD demonstrates strong detection timeliness, which is beneficial for defending against APT attacks.

4.6. Visualization analysis

In this section, we analyze the raw graph representations produced by CGL-AD for the three datasets. We use Principal Component Analysis (PCA) to visualize the representation spaces on a two-dimensional plane. Fig. 6 shows the PCA decomposition of the local and global representations at the final moment t_x for each graph in the Streamspot dataset. From the figure, we can see that both the local and global representation of the attack and normal graphs are almost linearly separable, even without further processing. Benign activities (i.e., YouTube, Gmail, VGame, Download and CNN) fall into several tight clusters, whereas abnormal activity (cluster in red) is well partitioned from these benign activities.

Compared to other datasets, the StreamSpot dataset is relatively simpler, which can be reflected in two aspects. Firstly, the average number of nodes in the StreamSpot dataset is only 3.18% of that in the Camflow-apt dataset; Secondly, there is a significant difference in the size between benign graphs and attack graphs. The average number of edges in benign graphs is 173,857, while in attack graphs, it is only 28,423. In contrast, the average number of edges in both benign and attack graphs in the Camflow-apt dataset is 975,226 and 957,968, respectively, with no significant difference. This makes it easier to discover anomalous patterns in the StreamSpot dataset.

We also visualize the representation for the Camflow-apt dataset in Fig. 7. Figs. 7(a)–7(c) show the local representation space. Figs. 7(d)–7(f) show the global representation space. Blue marks indicate representations from benign graphs, and red marks indicate representations from attack graphs. Figs. 7(a), 7(d) show the distribution of local and global representations of 150 provenance graphs in this dataset at t_0 , while Figs. 7(b), 7(e) correspond to the last moment t_x . From these figures, we can observe that the representations of benign and attack graphs are not well-separated, either initially or at the final time point. To further analyze this, we sample a benign graph G^1 and an attack graph G^2 , and visualize their local and global representations over time from t_0 to t_x in a two-dimensional plane (as shown in Figs. 7(c), 7(f)). From the trajectories of the representations, we can observe moments in the attack graph where the behavioral changes deviate noticeably from the normal mode (indicated by shaded regions). CGL-AD precisely captures these time-dependent changes in behavioral patterns through its sequence learning module, thereby improving the detection performance.

5. Conclusion

In this paper, we present CGL-AD, a novel contextualized graph learning framework designed for detecting Advanced Persistent Threats. CGL-AD addresses the limitation of static methods in capturing temporal information through dynamic graph learning. Additionally, CGL-AD's innovative combination of three advanced techniques enables it to capture complex structural patterns in provenance graphs that are often overlooked by existing methods. Specifically, CGL-AD utilizes a temporal graph network to learn global structural transformations over time, integrates a data stream frequency estimation technique to

identify local heterogeneous topology changes, and leverages sequence learning to comprehensively capture normal behavioral patterns. Our extensive experiments demonstrate that CGL-AD achieves high accuracy and low false alarm rates, while also enabling early detection of attacks, which is crucial for timely mitigation of potential threats. Overall, CGL-AD offers a practical and scalable solution for APT detection, improving upon existing methods.

Future work. The primary limitation of CGL-AD is its vulnerability to distribution shifts and adversarial attacks. In future work, we plan to: (1) enhance CGL-AD's robustness when handling out-of-distribution data; (2) investigate the sensitivity of CGL-AD to adversarial attacks, focusing on both designing and mitigating these attacks on input graphs or GNNs; and (3) explore more fine-grained detection methods, such as node- or edge-level analysis, for effective attack reconstruction.

CRedit authorship contribution statement

Lin Wang: Conceptualization, Methodology, Data curation, Formal analysis, Investigation, Software, Validation, Visualization, Writing – original draft. **Lanting Fang:** Conceptualization, Resources, Funding acquisition, Supervision, Validation, Writing – review & editing. **Yining Hu:** Validation, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China No. 6247073423, National Key Research and Development Program of China under Grant No. 2023YFC3010302.

Data availability

The Streamspot dataset comes from <https://github.com/sbustreamspot/sbustreamspot-data>. The Camflow-apt dataset and Shellshock dataset come from <https://github.com/margoseltzer>. And the code implementation will be made available on request.

References

Alsaheel, A., Nan, Y., Ma, S., Yu, L., Walkup, G., Celik, Z. B., et al. (2021). {ATLAS}: A sequence-based learning approach for attack investigation. In *30th USENIX security symposium* (pp. 3005–3022).

Chen, T., Dong, C., Lv, M., Song, Q., Liu, H., Zhu, T., et al. (2022). Apt-kgl: An intelligent apt detection system based on threat knowledge and heterogeneous provenance graph learning. *IEEE Transactions on Dependable and Secure Computing*.

Cheng, Z., Lv, Q., Liang, J., Wang, Y., Sun, D., Pasquier, T., et al. (2023). Kairos: Practical intrusion detection and investigation using whole-system provenance. arXiv preprint [arXiv:2308.05034](https://arxiv.org/abs/2308.05034).

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., et al. (2014). Learning phrase representations using rnn encoder–decoder for statistical machine translation. arXiv preprint [arXiv:1406.1078](https://arxiv.org/abs/1406.1078).

Dornhackl, H., Kadletz, K., Luh, R., & Tavalato, P. (2014). Malicious behavior patterns. In *2014 IEEE 8th international symposium on service oriented system engineering* (pp. 384–389). IEEE.

Gibbons, J. D., & Chakraborti, S. (2014). *Nonparametric statistical inference: revised and expanded*. CRC Press.

Han, X., Pasquier, T., Bates, A., Mickens, J., & Seltzer, M. (2020). Unicorn: Runtime provenance-based detector for advanced persistent threats. arXiv preprint [arXiv:2001.01525](https://arxiv.org/abs/2001.01525).

Hassan, W. U., Guo, S., Li, D., Chen, Z., Jee, K., Li, Z., et al. (2019). Nodozo: Combatting threat alert fatigue with automated provenance triage. In *Network and distributed systems security symposium*.

Hassan, W. U., Li, D., Jee, K., Yu, X., Zou, K., Wang, D., et al. (2020). This is why we can't cache nice things: Lightning-fast threat hunting using suspicion-based hierarchical storage. In *Annual computer security applications conference* (pp. 165–178).

Hossain, M. N., Milajerdi, S. M., Wang, J., Eshete, B., Gjomemo, R., Sekar, R., et al. (2017). {SLEUTH}: Real-time attack scenario reconstruction from {COTS} audit data. In *26th USENIX security symposium* (pp. 487–504).

Jenkinson, G., Carata, L., Balakrishnan, N., Bytheway, T., Sohan, R., Watson, R., et al. (2017). Applying provenance in apt monitoring and analysis. In *Proc. USENIX workshop theory practice provenance* (pp. 16–16).

Jia, Z., Xiong, Y., Nan, Y., Zhang, Y., Zhao, J., & Wen, M. (2023). Magic: Detecting advanced persistent threats via masked graph representation learning. arXiv preprint [arXiv:2310.09831](https://arxiv.org/abs/2310.09831).

Kapoor, M., Melton, J., Ridenhour, M., Krishnan, S., & Moyer, T. (2021). Prov-gem: Automated provenance analysis framework using graph embeddings. In *2021 20th IEEE international conference on machine learning and applications* (pp. 1720–1727). IEEE.

Lai, S., Xu, L., Liu, K., & Zhao, J. (2015). Recurrent convolutional neural networks for text classification. In *Proceedings of the AAAI conference on artificial intelligence*.

Liu, Y., Zhang, M., Li, D., Jee, K., Li, Z., Wu, Z., et al. (2018). Towards a timely causality analysis for enterprise security. In *NDSS*.

Manzoor, E., Milajerdi, S. M., & Akoglu, L. (2016). Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1035–1044).

Milajerdi, S. M., Eshete, B., Gjomemo, R., & Venkatakrishnan, V. (2019). Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security* (pp. 1795–1812).

Milajerdi, S. M., Gjomemo, R., Eshete, B., Sekar, R., & Venkatakrishnan, V. (2019). Holmes: real-time apt detection through correlation of suspicious information flows. In *2019 IEEE symposium on security and privacy* (pp. 1137–1152). IEEE.

Pasquier, T., Han, X., Goldstein, M., Moyer, T., Eysers, D., Seltzer, M., et al. (2017). Practical whole-system provenance capture. In *Proceedings of the 2017 symposium on cloud computing* (pp. 405–418).

Paudel, R., & Huang, H. H. (2022). Pikachu: Temporal walk based dynamic graph embedding for network anomaly detection. In *NOMS 2022-2022 IEEE/IFIP network operations and management symposium* (pp. 1–7). IEEE.

Pohly, D. J., McLaughlin, S., McDaniel, P., & Butler, K. (2012). Hi-fi: collecting high-fidelity whole-system provenance. In *Proceedings of the 28th annual computer security applications conference* (pp. 259–268).

Prasad, V., Cohen, W., Eigler, F., Hunt, M., Keniston, J., & Chen, B. (2005). Locating system problems using dynamic instrumentation. In *2005 ottawa linux symposium* (pp. 49–64). New York, NY: IEEE.

Rehman, M. U., Ahmadi, H., & Hassan, W. U. (2024). Flash: A comprehensive approach to intrusion detection via provenance graph representation learning. In *2024 IEEE symposium on security and privacy* (p. 139). IEEE Computer Society.

Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., & Bronstein, M. (2020). Temporal graph networks for deep learning on dynamic graphs. arXiv preprint [arXiv:2006.10637](https://arxiv.org/abs/2006.10637).

Shervashidze, N., Schweitzer, P., Van Leeuwen, E. J., Mehlhorn, K., & Borgwardt, K. M. (2011). Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12.

Shi, Y., Huang, Z., Feng, S., Zhong, H., Wang, W., & Sun, Y. (2020). Masked label prediction: Unified message passing model for semi-supervised classification. arXiv preprint [arXiv:2009.03509](https://arxiv.org/abs/2009.03509).

Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). *Mitre att & ck: Design and philosophy: Technical report*, The MITRE Corporation.

Wagner, M., Fischer, F., Luh, R., Haberson, A., Rind, A., Keim, D. A., et al. (2015). A survey of visualization systems for malware analysis. In *Eurographics conference on visualization* (pp. 105–125). The Eurographics Association.

Wang, Q., Hassan, W. U., Li, D., Jee, K., Yu, X., Zou, K., et al. (2020). You are what you do: Hunting stealthy malware via data provenance analysis. In *NDSS*.

Wang, S., Wang, Z., Zhou, T., Sun, H., Yin, X., Han, D., et al. (2022). Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security*, 17, 3972–3987.

Xie, Y., Feng, D., Tan, Z., & Zhou, J. (2016). Unifying intrusion detection and forensic analysis via provenance awareness. *Future Generation Computer Systems*, 61, 26–36.

Xiong, C., Zhu, T., Dong, W., Ruan, L., Yang, R., Cheng, Y., et al. (2020). Conan: A practical real-time apt detection system with high accuracy and efficiency. *IEEE Transactions on Dependable and Secure Computing*, 19, 551–565.

Yadav, T., & Rao, A. M. (2015). Technical aspects of cyber kill chain. In *Security in computing and communications: third international symposium, SSCC 2015, Kochi, India, August 10-13, 2015. proceedings 3* (pp. 438–452). Springer.

Yang, D., Li, B., Rettig, L., & Cudré-Mauroux, P. (2017). Histosketch: Fast similarity-preserving sketching of streaming histograms with concept drift. In *2017 IEEE international conference on data mining* (pp. 545–554). IEEE.

Yang, F., Xu, J., Xiong, C., Li, Z., & Zhang, K. (2023). {PROGRAPHER}: An anomaly detection system based on provenance graph embedding. In *32nd USENIX security symposium* (pp. 4355–4372).

Zeng, J., Wang, X., Liu, J., Chen, Y., Liang, Z., Chua, T. S., et al. (2022). Shadewatcher: Recommendation-guided cyber threat analysis using system audit records. In *2022 IEEE symposium on security and privacy* (pp. 489–506). IEEE.

Zipperle, M., Gottwalt, F., Chang, E., & Dillon, T. (2022). Provenance-based intrusion detection systems: A survey. *ACM Computing Surveys*, 55, 1–36.